

Supplementary Material

Public repository: <https://github.com/gabrielmontufar/article-114-optimization-and-engineering-claim-admissibility>. This repository contains Online Resource 1, Online Resource 2, and the public-facing README for the article 114 Optimization and Engineering package. Third-party raw datasets are not bundled in Online Resource 2; the repository provides processed response-support points, manifests, acquisition notes, and acquisition instructions.

Supplementary Material for Optimizer-Agnostic Claim-Admissibility for Discrete Structural Design Optimization Outputs

Gabriel Jesús Montúfar Chiriboga

This file contains only supplementary methods, extended evidence, replication notes, reporting-guide alignment, and the certificate-handoff schema used to connect the optimizer-agnostic encoded-claim layer to future validation workflows. Candidate-level data, figures, executable scripts, `run_all_log.txt`, `cgc_claim_risk_sensitivity.csv`, `cgc_certificate_handoff_schema.json`, and environment files are supplied in the submitted clean ZIP package and the public supplementary repository.

S1. Mathematical scope and encoded-claim notation

The certificate is conditional: a design supports only the requirements encoded in the declared residual vector, catalog-membership checks, tolerance policy, and verification route. It is not a professional approval, comprehensive experimental validation, complete regulatory approval, or unrestricted global MINLP certificate.

A candidate x supports an encoded-compliance claim only when every encoded residual is non-positive within tolerance, catalog membership is valid, and the solver/enumeration status is recorded. If any condition is absent, the supported statement is downgraded to an optimization-result claim or an unresolved-claim record. Table S1 summarizes the certificate components, the evidence required for each component, and the type of unsupported overstatement that each check prevents.

Table S1. Certificate components, required evidence, and unsupported overstatements prevented.

Certificate component | Required evidence | Unsupported overstatement prevented

Residual vector | Named encoded requirements and tolerance | Feasible-looking design reported as compliant without checks

Catalog membership | Discrete section/area index from declared catalog | Continuous or off-catalog design reported as buildable

Verification route | Enumeration, solver status, or independent evaluator log | Untraceable claim about why the design is acceptable

Scope limits | Explicit unencoded requirements | Nominal encoded feasibility reported as physical safety or full building-code approval

S2. Benchmark definitions and data files

The benchmark suite is split between the manuscript summary and machine-readable files. The complete candidate tables are intentionally kept as CSV files to avoid unreadable multi-column tables in the Word/PDF submission. Table S2 lists the benchmark files used in the supplementary package, their validation role, and the main output produced by each file.

The file `cgc_operator_property_examples.csv` provides a compact index of benchmark-level examples for the post-optimization admissibility operator properties used in the manuscript: missing-evidence blocking, compatible-evidence monotonicity and contradictory-evidence downgrade, idempotence for an unchanged evidence record, and scope monotonicity. It is an index over existing benchmark outputs rather than a new structural validation result.

Table S2. Benchmark files, validation roles, and main outputs.

Benchmark file | Role in validation | Main output

`cgc_scalar_benchmark.py` / `.csv` | Minimal falsifiable example of residual-positive penalty selection | Claim-risk status for scalar feasible and infeasible choices

cgc_ten_bar_truss_exhaustive_grouped.py / .csv | Exact grouped finite-catalog truss enumeration | 100,000 candidates and 3,518 feasible designs

cgc_milp_crosscheck_pulp.py / log | Independent MILP selection over pre-enumerated feasible candidates | Minimum selection within the enumerated feasible set

cgc_opensees_blind_crosscheck.py / .csv | Independent evaluator check | 0/45 classification mismatches

cgc_aisc_portal_frame_catalog_benchmark.py / .csv | Real W-shape catalog audit | 83,521 member-pair candidates and 37,156 feasible candidates

cgc_uncertainty_robustness_probe.py / .csv | Boundary robustness probe | Nominal feasibility does not imply robust-compliance claim

S3. Penalty-weight and claim-risk reporting

Penalty-weight sweeps are reported to show that penalties can reduce residuals without authorizing encoded-compliance wording. The claim-risk benchmark now uses a deterministic ordinal rule as the primary classification: positive residual or unsupported encoded feasibility yields high claim risk; zero residual plus encoded feasibility but no recorded certificate yields moderate claim risk; zero residual plus encoded feasibility plus recorded enumeration or solver route yields low encoded-claim risk. The numeric score is retained only as a secondary audit index, and cgc_claim_risk_sensitivity.csv reports baseline, residual-heavy, certificate-heavy, and feasibility-heavy weight sets. Table S3 identifies the reported penalty-weight and claim-risk quantities and explains why each quantity is included in the audit record.

Table S3. Penalty-weight and claim-risk reporting quantities.

Reported quantity | Reason for inclusion

Objective value | Shows optimization performance within the tested formulation

Maximum positive residual | Separates true encoded feasibility from residual-positive outcomes

Catalog status | Confirms whether the design belongs to the declared discrete catalog

Claim label | Prevents a residual-positive or out-of-scope result from being described as compliant

S4. Independent OpenSeesPy blind FEM cross-check

The independent OpenSeesPy cross-check evaluates 45 cases with a separate finite-element implementation. The result file reports zero classification mismatches and numerical differences at round-off scale, supporting the structural evaluator used in the certification workflow.

S5. AISC W-shape real-catalog portal-frame audit

The AISC benchmark is an optional external-data benchmark. The AISC spreadsheet is not redistributed in the clean ZIP; users must obtain the official spreadsheet and place it at the documented path. The local audit used 289 W-shapes, producing 83,521 member-pair candidates and 37,156 feasible candidates under the declared simplified loads and encoded limits. The file cgc_aisc_source_traceability.csv reports the local spreadsheet SHA256 used for verification, the benchmark-output SHA256, shape count, candidate count, feasible count, and best feasible W-shape pair. This supports only an AISC W-shape catalog audit under declared assumptions, not AISC approval, professional certification, comprehensive experimental validation, or full building-code translation.

S6. Certificate propositions and scope limits

Proposition S1, encoded-compliance certificate: if all declared encoded residuals are non-positive within tolerance, catalog membership is valid, and the verification route is recorded, the design supports only the corresponding encoded-compliance claim.

Proposition S2, finite-catalog optimality certificate: if every member of a finite declared catalog set is evaluated and the selected candidate has the minimum objective among feasible candidates, optimality is certified only over that finite enumerated set.

Proposition S3, penalty non-certification: a penalty-selected candidate with any positive encoded residual cannot support an encoded-compliance claim even if its penalized objective is competitive.

S7. Monte Carlo robustness boundary probe

The robustness probe perturbs load and elastic modulus by $\pm 10\%$ over 2,000 samples per design. The nominal exact design had feasible rate 0.5515 and the near-boundary design had feasible rate 0.3675. These results deliberately prevent nominal encoded feasibility from being overstated as a robust-design-support claim.

S8. Alignment with original-research reporting guides

The reporting package follows original-research principles from the supplied ICMJE, MDAR, EQUATOR, and LEEME guidance: transparent methods, explicit materials/data availability, reproducible computational workflow, limits of inference, and separation between main findings and supplementary evidence. Table S4 summarizes how the supplementary package aligns reporting items with the locations where they are addressed in the manuscript and supporting files.

Table S4. Reporting-guide alignment items and locations.

Reporting item | Where addressed

Methods transparency | Main methods plus S1-S3 and executable scripts

Verification, response-support sensitivity, and reproducibility | OpenSeesPy cross-check, AISC optional audit and traceability, threshold-sensitivity table, `run_all.py --quick/--smoke`, README, requirements.txt

Limits of inference | Main limitations plus S5-S7 scope statements

Data/material availability | CSV files, scripts, AISC acquisition note, clean supplementary ZIP

S9. Certificate handoff for virtual testing and full-code audit

The file `cgc_certificate_handoff_schema.json` provides a structured example of the information that a residual-evidence claim passport should pass to downstream verification environments: geometry, assigned catalog sections, encoded residual checks, declared exclusions, verification route, and intended handoff targets. The schema is deliberately not presented as evidence of completed external response validation or clause-complete code support. Its role is to make the transition from optimization audit to later nonlinear virtual testing, commercial design-engine checking, or laboratory-response planning explicit and reproducible.

A downstream virtual-laboratory implementation could import the certificate into a high-fidelity nonlinear finite-element model with material inelasticity, geometric nonlinearity, post-buckling checks, or dynamic collapse simulations. A downstream full-code implementation could map the same metadata into a commercial design-checking API and return violated clauses as new residuals. These extensions are future validation pathways, not claims made by the present benchmark.

S10. Code-and-response support extension

The files `cgc_full_code_clause_register.csv`, `cgc_partial_code_oracle_comparison.py`, `cgc_portal_frame_clause_support_gate.py`, `cgc_extended_claim_risk_benchmark.py`, and `cgc_aisc_claim_gate.py` add explicit clause-support and code-oracle status fields to the benchmark report. The current status is partial code-oracle support for declared simplified checks, not full ASCE/AISC approval. A full-code-check-supported claim would require clause-complete coverage for the declared code editions, load combinations, member types, and connection/detailing assumptions, plus independent oracle agreement.

The evidence ladder is L0 optimization-result only; L1 encoded-compliance support; L2 load-code-supported support; L3 clause-complete code-check-supported support; L4 published-response support within a declared structural class and loading protocol; and L5 engineering approval. L5 is outside the paper. L3 and L4 are admissible only when the corresponding oracle and published-response evidence files report passing status.

The supplementary file `cgc_code_physics_validation_extension.md` records this ladder, required evidence, current package status, and blocked overclaim examples so that reviewers can distinguish encoded feasibility, partial code-oracle support, clause-complete code-check support, and published-response support.

The files `cgc_portal_frame_published_test_register.csv`, `cgc_portal_frame_experimental_response.csv`, `cgc_portal_frame_response_support.py`, `cgc_portal_frame_response_support_summary.csv`, `cgc_portal_frame_response_support_details.csv`, `cgc_response_support_threshold_sensitivity.csv`, `cgc_portal_frame_raw_data_manifest.csv`, and `cgc_portal_frame_raw_data_acquisition_note.md` add a limited published-response support layer. The raw DesignSafe/NEES SAC Steel Project Experiment 5 files are not bundled in the submission ZIP; acquisition instructions are provided, while the processed service-envelope points are included in the ZIP. The script screens ten raw cyclic response curves, accepts seven specimens under the declared service-response and initial-stiffness tolerances, logs three failed or insufficient cases, and reports `false_certified_count = 0`. The sensitivity table reports how the passing count changes under NRMSE limits 0.10/0.15/0.20 and stiffness-error limits 0.15/0.20/0.25. The resulting `response_support_status` is `supported_by_processed_published_response_data` for the tested steel-frame response family only; it is not safety certification, AISC approval, professional certification, or complete regulatory approval.

The files `cgc_truss_published_test_register.csv`, `cgc_truss_experimental_response.csv`, `cgc_truss_response_support.py`, `cgc_truss_response_support_summary.csv`, `cgc_truss_response_support_details.csv`, `cgc_response_support_threshold_sensitivity.csv`, `cgc_truss_raw_data_manifest.csv`, and `cgc_truss_raw_data_acquisition_note.md` add a limited published truss published-response support layer. The public Zenodo workbook 10.5281/zenodo.15658671 is not bundled in the submission ZIP; the ZIP provides processed response points, a manifest, and acquisition instructions for optional raw-data reruns. The module screens seven truss damage/collapse cases against intact-state sensor baselines, accepts six under the declared detection rule, logs one discrepancy, and reports `false_certified_count = 0`. The sensitivity table reports the passing count under detection-rate limits 0.45/0.50/0.55.

S11. Reviewer-facing claim-status controls

The final supplementary package adds reviewer-facing controls that test whether the framework blocks unsupported wording rather than merely accepting residual-zero cases. The file `cgc_claim_status_by_evidence_layer.csv` records the evidence layer supporting each benchmark claim. The negative-control claim test deliberately introduces off-catalog sections, positive residuals, missing verification routes, AISC-approval wording without code-clause evidence, and robust wording without robust-design evidence; all cases are blocked or downgraded and `false_certified_count` remains 0. The manual oracle check compares 18 hand-calculation cases for drift, demand/capacity, strong-column/weak-beam, shear hierarchy, catalog membership, and residual sign convention, with zero classification mismatches. The file `cgc_reviewer_claim_audit_checklist.md` maps likely reviewer questions to the exact supplementary artifact that answers each question.

S12. Software-package integrity.

The files `cgc_software_package_integrity.py`, `cgc_software_package_integrity_details.csv`, `cgc_software_package_integrity_summary.csv`, and `cgc_software_package_integrity_note.txt` check the reproducibility package across presentation, application, data, and integration layers. The checker compiles package Python scripts, checks required entry points, verifies critical CSV schemas and processed datasets, confirms ZIP exclusions and upload-package hash consistency, and writes `software_package_integrity_status` back to `cgc_benchmark_reporting_summary.csv`. A passed result authorizes only software-package integrity of the reproducibility package; it is not structural-code approval, professional approval, legal approval, or complete regulatory approval. The checker also confirms that processed response-support points, response-support threshold-sensitivity tables, AISC traceability outputs, acquisition notes for nonbundled raw sources, and package dependency instructions are present when the full package is checked.

Third-party raw-data redistribution note. The file `third_party_raw_data_redistribution_note.md` summarizes the package boundary for third-party raw sources. The Zenodo truss workbook is public and citable, but it is not bundled in the submission ZIP; reviewers can obtain it from the public Zenodo record for optional raw-data reruns. The DesignSafe/NEES portal-frame raw files and the AISC Shapes Database spreadsheet are also not redistributed in the submission ZIP. The package therefore includes processed response-support points, source manifests, acquisition notes, AISC source/output checksums, and rerun instructions instead.